

Docker y Kubernetes

Ciclo de vida del Software







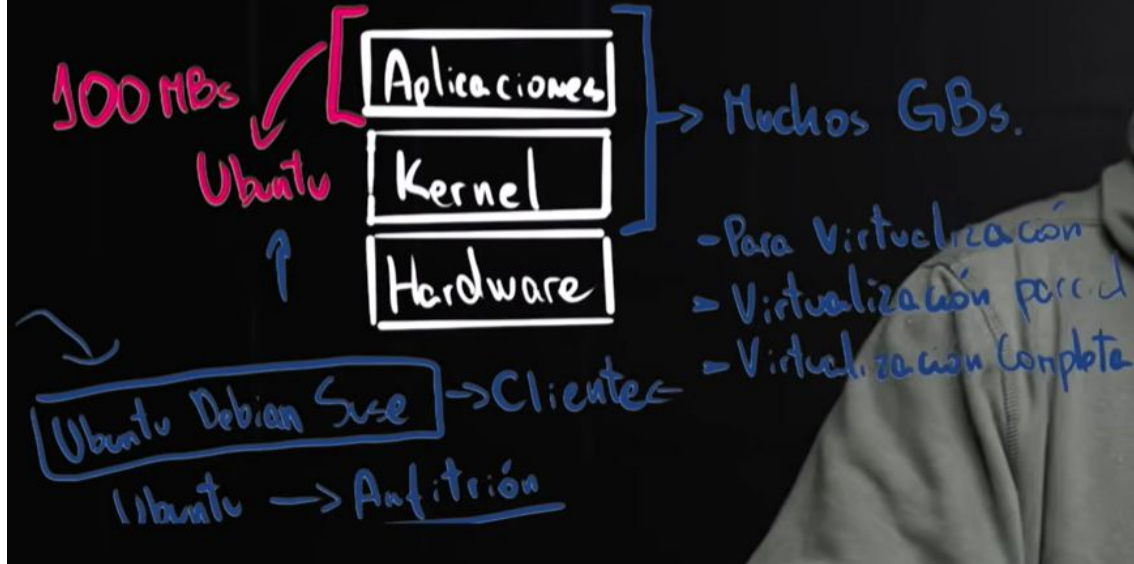
docker

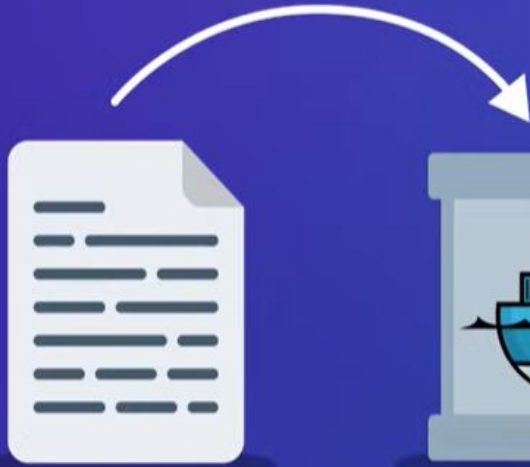
Virtualización

Docker

VS

Virtual
Machine
VM

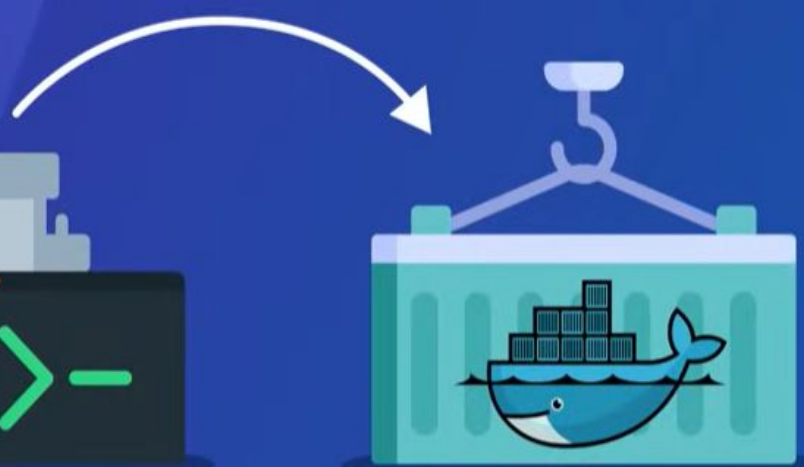




DOCKER FILE



DOCKER IMAGE

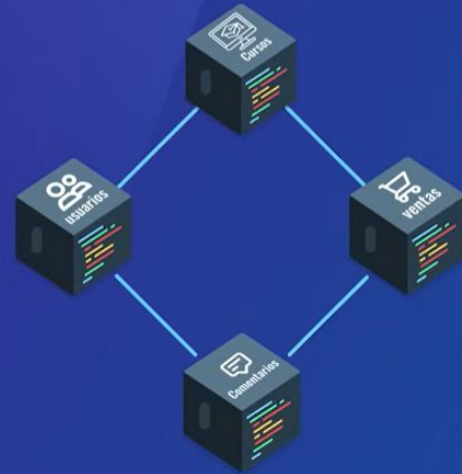


DOCKER CONTAINER



MONOLÍTICA

ARQUITECTURA DE MICROSERVICIOS









docker



DOCKER FILE



Kubernetes



CONTROL PLANE

Herramientas Docker

- Docker Desktop
- Docker Hub
- CLI
 - Comandos: `docker images`, `docker pull node`, `docker pull node:18`, `docker pull mysql`, `docker image rm node`, `docker {container} create mongo` (este comando devuelve un id y con ese id puedo inicializar el contenedor), `docker start [id]`, `docker stop [id]`, `docker ps` (muestra contenedores ejecutandose), `docker ps -a` (muestra todos incluso los que no se ejecutan)

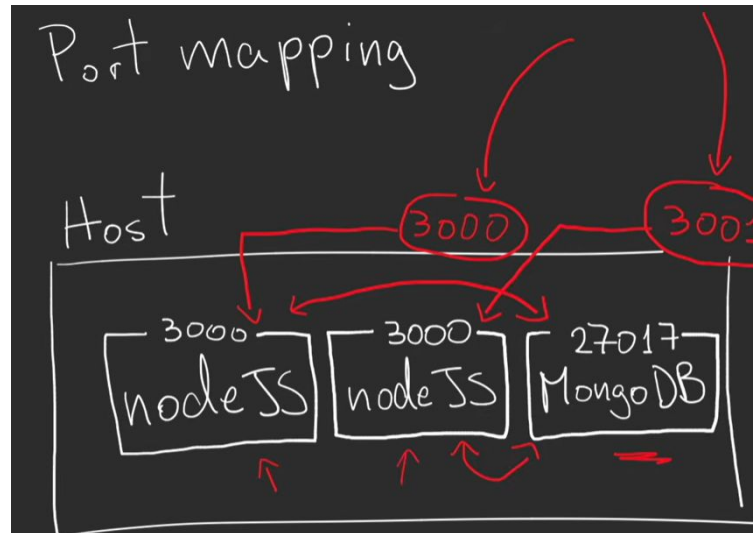
```
~ > docker pull mongo
Using default tag: latest
latest: Pulling from library/mongo
11e23ac719b3: Pull complete
566a81ecfaf7: Pull complete
82cf8516df40: Pull complete
4334b4723aa2: Pull complete
890407c66151: Pull complete
df35225c33e7: Pull complete
37bf28fa4f66: Pull complete
c7f3968f2dd4: Pull complete
22d12b77d224: Pull complete
Digest: sha256:37e84d3dd30cdfb5472ec42b8a6b4dc6ca7cacd91ebcfa0410a54528bbc5fa6d
Status: Downloaded newer image for mongo:latest
docker.io/library/mongo:latest
~ > docker create mongo
bedeedfd10cab297dc18594e273612f46b6fdb65af26b81eddc8fb5a8a0dcaf
~ > docker container create mongo
~ > docker start bedeedfd10cab297dc18594e273612f46b6fdb65af26b81eddc8fb5a8a0dcaf
bedeedfd10cab297dc18594e273612f46b6fdb65af26b81eddc8fb5a8a0dcaf
~ > docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
bedeedfd10cab297dc18594e273612f46b6fdb65af26b81eddc8fb5a8a0dcaf	mongo	"docker-entrypoint.s..."	About a minute ago	Up 15 seconds	27017/tcp	stoic

```
~ >
```

```
bedeedfd10ca ~ > docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES
bedeedfd10ca   mongo    "docker-entrypoint.s..." 4 minutes ago  Exited (0) 47 seconds ago
~ > docker rm stoic_stonebraker
stoic_stonebraker
```

- `docker create --name monguito mongo` (para crear un contenedor con un nombre que yo elijo)



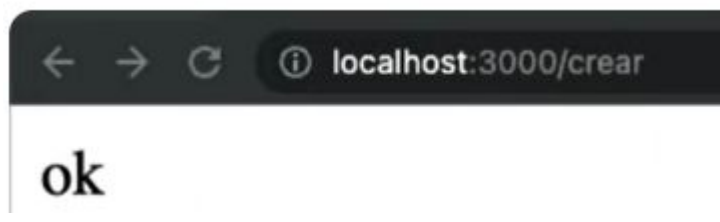
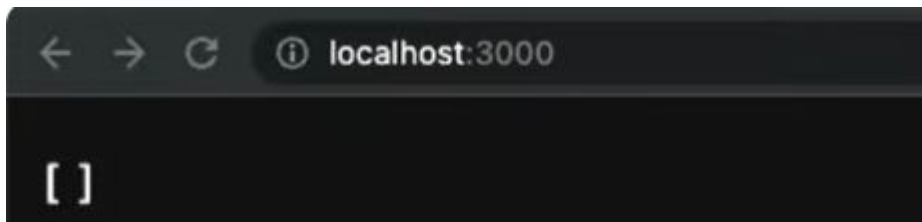
Mapear puertos

- `docker create -p27017:27017 --name monguito mongo (p:publish/puerto, primer valor puerto de mi máquina externa, segundo valor puerto de mi contenedor)`
- `docker logs [nombre del contenedor]` para corroborar si mi puerto se mapeo correctamente.
- `docker run mongo` (hace 3 cosas, primero busca Imagen, si no la encuentra la descarga, Crea el contenedor y luego Inicia el contenedor)

```
index.js X
1 import express from 'express'
2 import mongoose from 'mongoose'
3
4 const Animal = mongoose.model('Animal', new mongoose.Schema({
5   tipo: String,
6   estado: String,
7 })))
8
9 const app = express()
10
11 mongoose.connect('mongodb://nico:password@localhost:27017/miaapp?authSource=admin')
12
13 app.get('/', async (_req, res) => {
14   console.log('listando...')
15   const animales = await Animal.find();
16   return res.send(animales)
17 })
18 app.get('/crear', async (_req, res) => {
19   console.log('creando...')
20   await Animal.create({ tipo: 'Chanchito', estado: 'Feliz' })
21   return res.send('ok')
22 })
23
24 app.listen(3000, () => console.log('listening...'))
```

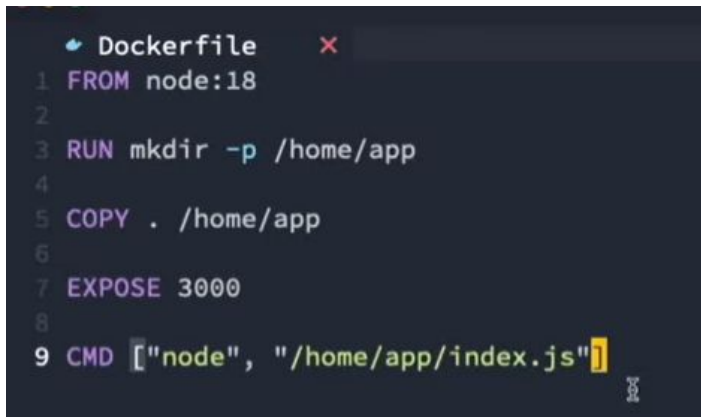
Configurar Container

- docker pull mongo (traigo la imagen)
- docker create -p27017:27017 monguito -e MONGO_INITDB_ROOT_USERNAME=marcos -e MONGO_INITDB_ROOT_PASSWORD=password mongo
- docker start monguito
- docker ps (muestra container corriendo)
- node [index.js](#) (corro la aplicacion)
- En el navegador ir a localhost:3000 y localhost:3000/crear



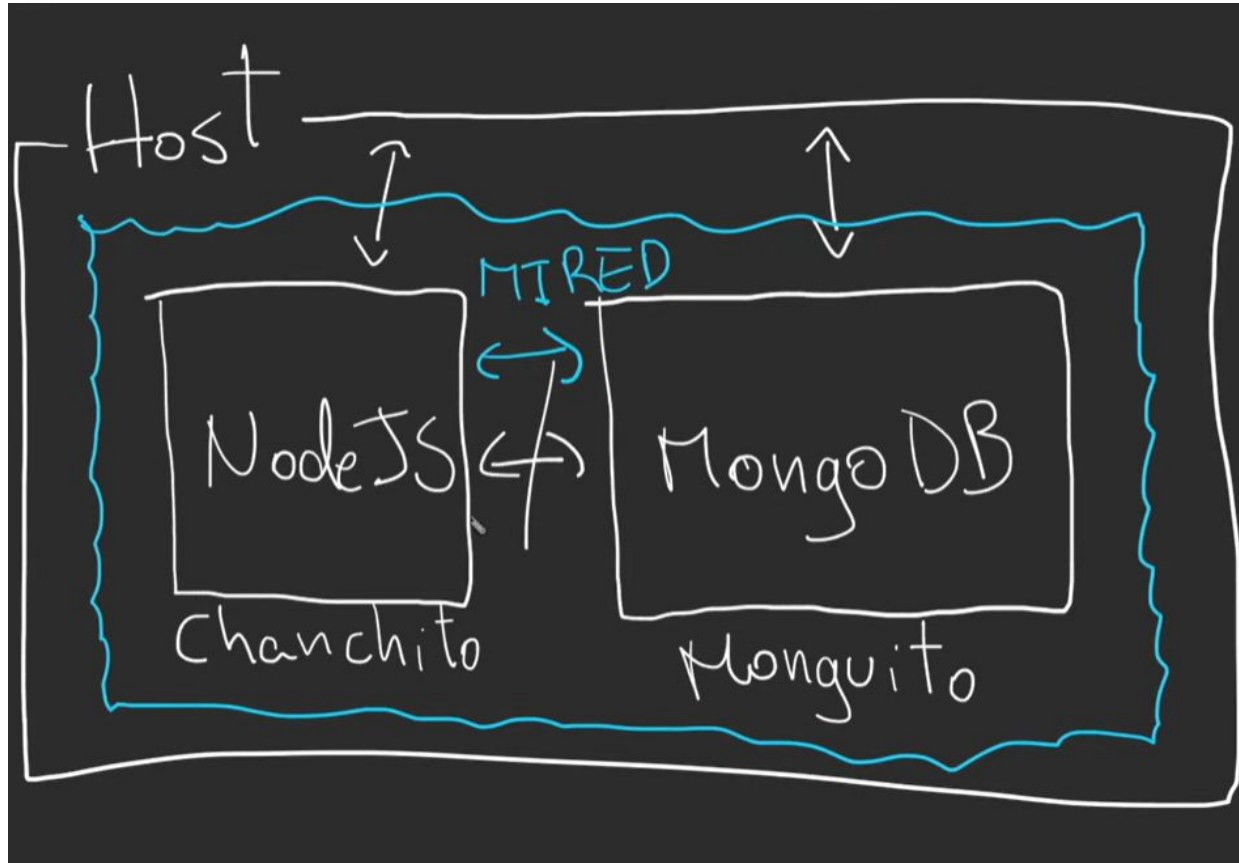
Como llevar mi aplicación a un container

- En el repositorio de mi aplicación crear archivo Dockerfile (no puede tener otro nombre y no lleva ninguna extensión el archivo)
- Escribir en el archivo

A screenshot of a code editor showing a Dockerfile. The editor has a dark background with light-colored text. The title bar of the editor shows 'Dockerfile' with a blue icon on the left and a red 'X' on the right. The code is as follows:

```
1 FROM node:18
2
3 RUN mkdir -p /home/app
4
5 COPY . /home/app
6
7 EXPOSE 3000
8
9 CMD ["node", "/home/app/index.js"]
```

The line numbers 1 through 9 are on the left side of the code. The code uses color-coding: 'FROM' is purple, 'RUN' is blue, 'COPY' is purple, 'EXPOSE' is purple, and 'CMD' is blue. The string literals are in green. There is a small yellow cursor at the end of line 9.



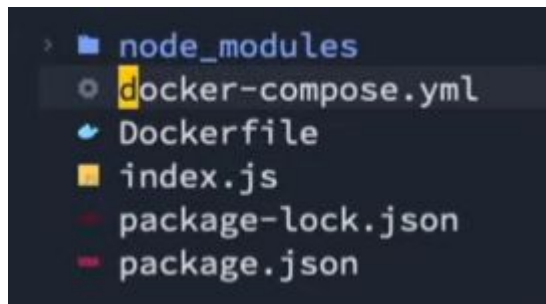
Crear red en docker

- `docker network ls` (para ver todas las redes existentes)
- `docker network create mired`
- `docker build -t miapp:latest | 1` (crea la imagen)
- `docker rm monguito` (eliminar mi contenedor para crearlo dentro de mired)
- `docker create -p27017:27017 --name monguito --network mired -e MONGO_INITDB_ROOT_USERNAME=marcos -e MONGO_INITDB_ROOT_PASSWORD=password mongo`
- `docker create -p3000:3000 --name chanchito --network mired miapp:1`
- Arrancar ambos contenedores: `docker start [nombre del contenedor]`

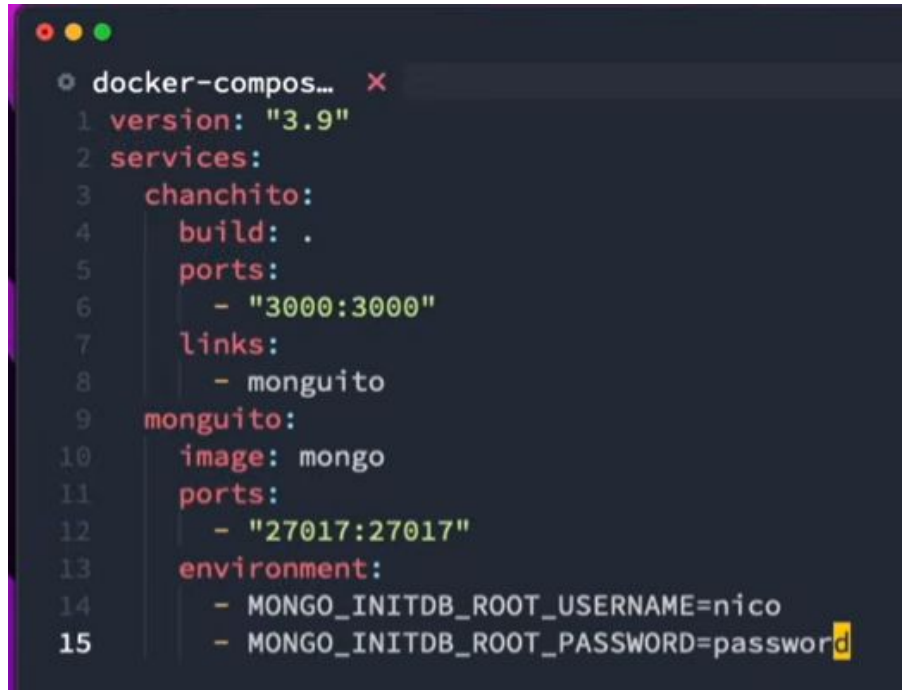
```
10  
11 mongoose.connect('mongodb://nico:password@monguito:27017/miapp?authSource=admin')  
12
```

Docker Compose

- Pasos hechos hasta ahora
 - Descargar imagen
 - Crear una red
 - Crear un contenedor
 - asignar puertos
 - asignar nombres
 - asignar variables de entorno
 - especificar red
 - indicar imagen

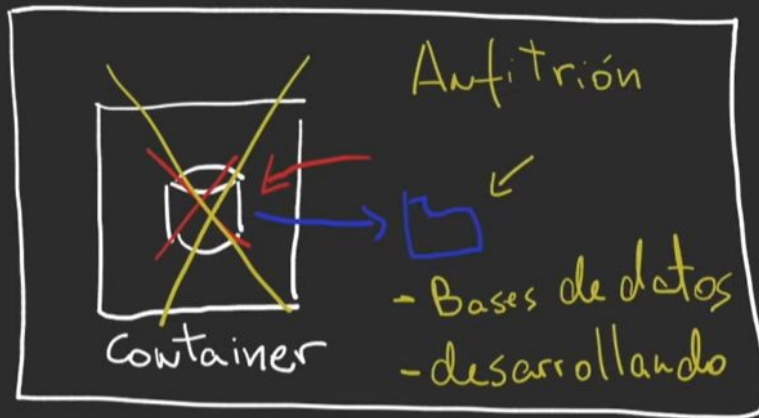


docker compose up/down

A screenshot of a terminal window with a dark background and light-colored text. The window title is "docker-compos..." with a red 'x' icon. The content is a Docker Compose configuration file with line numbers 1 through 15 on the left. The configuration defines two services: "chanchito" and "monguito". "chanchito" is built from the current directory, maps port 3000 to 3000, and links to "monguito". "monguito" uses the "mongo" image, maps port 27017 to 27017, and has environment variables for MONGO_INITDB_ROOT_USERNAME and MONGO_INITDB_ROOT_PASSWORD.

```
1 version: "3.9"
2 services:
3   chanchito:
4     build: .
5     ports:
6       - "3000:3000"
7     links:
8       - monguito
9   monguito:
10    image: mongo
11    ports:
12      - "27017:27017"
13    environment:
14      - MONGO_INITDB_ROOT_USERNAME=nico
15      - MONGO_INITDB_ROOT_PASSWORD=password
```

Volumes



Tipos

- Anónimo: solo indicas la ruta
- De Anfitrión o host
- Nombrado

```
❖ docker-compos... •
1 version: "3.9"
2 services:
3   chanchito:
4     build: .
5     ports:
6       - "3000:3000"
7     links:
8       - monguito
9   monguito:
10    image: mongo
11    ports:
12      - "27017:27017"
13    environment:
14      - MONGO_INITDB_ROOT_USERNAME=nico
15      - MONGO_INITDB_ROOT_PASSWORD=password
16    volumes:
17      - mongo-data:/data/db
18      # mysql -> /var/lib/mysql
19      # postgres -> /var/lib/postgresql/data
20
21 volumes:
22   mongo-data:
```

Múltiples ambientes

```
Dockerfile
1 FROM node:18
2
3 RUN mkdir -p /home/app
4
5 COPY . /home/app
6
7 EXPOSE 3000
8
9 CMD ["node", "/home/app/index.js"]
```

```
Dockerfile.dev Dockerfile
1 FROM node:18
2
3 RUN npm i -g nodemon
4 RUN mkdir -p /home/app
5
6 WORKDIR /home/app
7
8 EXPOSE 3000
9
10 CMD ["nodemon", "index.js"]
```


docker compose -f docker-compose-dev.yml

```
docker-compose-dev.yml
1 version: "3.9"
2 services:
3   chanchito:
4     build: .
5     ports:
6       - "3000:3000"
7     links:
8       - monguito
9   monguito:
10    image: mongo
11    ports:
12      - "27017:27017"
13    environment:
14      - MONGO_INITDB_ROOT_USERNAME=nico
15      - MONGO_INITDB_ROOT_PASSWORD=password
16    volumes:
17      - mongo-data:/data/db
18      # mysql -> /var/lib/mysql
19      # postgres -> /var/lib/postgresql/data
20
21 volumes:
22   mongo-data:
```

```
Dockerfile.dev  X  Dockerfile  X  docker-compose-dev.yml
1 version: "3.9"
2 services:
3   chanchito:
4     build:
5       context: .
6       dockerfile: Dockerfile.dev
7     ports:
8       - "3000:3000"
9     links:
10      - monguito
11    volumes:
12      - ../home/ap
13   monguito:
14     image: mongo
15     ports:
16       - "27017:27017"
17     environment:
18       - MONGO_INITDB_ROOT_USERNAME=nico
19       - MONGO_INITDB_ROOT_PASSWORD=password
20     volumes:
21       - mongo-data:/data/db
22       # mysql -> /var/lib/mysql
23       # postgres -> /var/lib/postgresql/data
24
25 volumes:
26   mongo-data:
```